

DOCUMENTO TÉCNICO

Sistema de Totem de Autoatendimento para Fast Food

Visão, Arquitetura, Segurança, Modelagem, Fluxos, APIs e Guia de Desenvolvimento

Informação	Descrição
Tipo de documento	Documento técnico de desenvolvimento de sistemas
Produto	Totem de autoatendimento para fast food
Stack recomendada	Java 21, Spring Boot, PostgreSQL, Angular ou React, PWA
Escopo	MVP com cardápio, pedidos, pagamento simulado, caixa, cozinha, admin e segurança
Versão	1.0 - Estrutura inicial para desenvolvimento

Sumário

1. Histórico e objetivo do documento
2. Resumo executivo
3. Problema e objetivos do produto
4. Escopo do MVP
5. Atores, perfis e dispositivos
6. Diagramas de casos de uso
7. Arquitetura geral e camadas
8. Segurança e controle de acesso
9. Ativação de dispositivos
10. Módulos do sistema
11. Modelagem UML e banco de dados
12. Estados, enums e regras de negócio
13. Pagamento e estratégia de integração
14. Fluxogramas e diagramas de sequência
15. Organização do backend
16. Endpoints REST
17. Contratos de dados
18. Boas práticas, testes e qualidade
19. Comprovante e modo totem
20. Roadmap, riscos e decisões pendentes
21. Conclusão técnica

1. Histórico e objetivo do documento

Este documento reorganiza a visão do Sistema de Totem de Autoatendimento para Fast Food em uma estrutura técnica voltada ao desenvolvimento. A proposta é transformar a ideia funcional em artefatos de engenharia: arquitetura, requisitos, segurança, modelagem, fluxos, APIs, entidades, regras de negócio, testes e roadmap.

O material deve servir como base para alinhamento entre cliente, desenvolvedor, equipe técnica e stakeholders, reduzindo ambiguidades antes da implementação do MVP.

- Definir a visão técnica do produto.
- Orientar backend, frontend, banco de dados e segurança.
- Registrar decisões arquiteturais iniciais.
- Mapear riscos e pontos de atenção.
- Oferecer diagramas úteis para desenvolvimento e apresentação técnica.

2. Resumo executivo

O sistema consiste em uma solução de autoatendimento para fast foods, permitindo que o cliente faça pedidos diretamente em um tablet, totem ou tela touch. O cliente visualiza o cardápio, seleciona produtos, personaliza itens, escolhe a forma de pagamento, recebe uma senha ou comprovante e acompanha o status do pedido.

A solução é composta por um frontend de totem, backend Java/Spring Boot, banco de dados relacional, painel do caixa, painel da cozinha, painel administrativo, módulo de segurança, módulo de dispositivos, auditoria e camada de pagamento desacoplada.

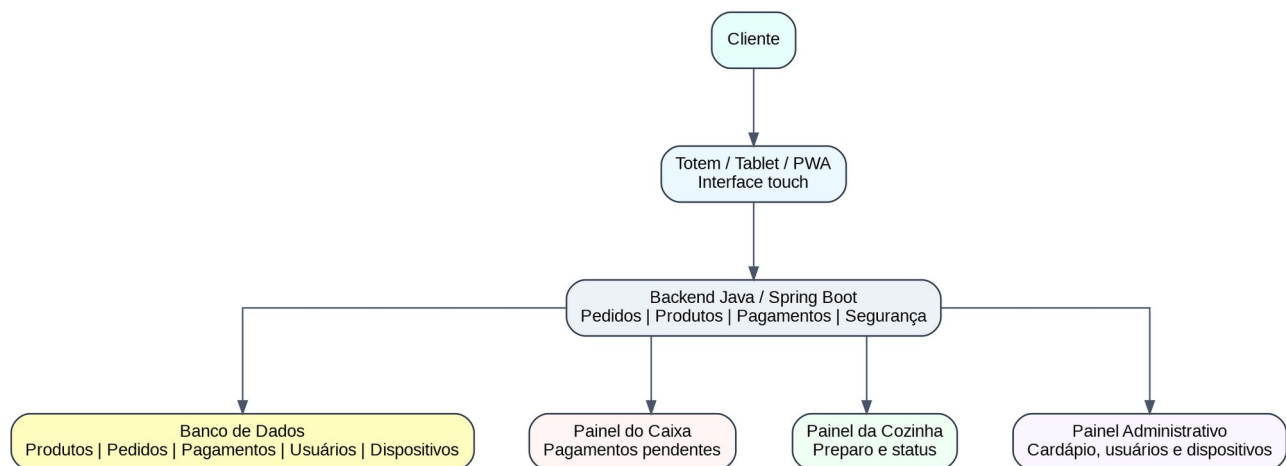


Figura 1 - Visão geral da solução e comunicação entre módulos.

3. Problema e objetivos do produto

3.1 Problema que o sistema resolve

Em operações de fast food, filas no caixa, dependência total de atendentes e falhas de comunicação podem gerar lentidão, pedidos incorretos e perda de produtividade. O totem reduz a sobrecarga do caixa e padroniza a coleta do pedido.

3.2 Benefícios

Beneficiário	Benefícios
Cliente	Autonomia, visualização clara do cardápio, personalização, menor tempo em fila e acompanhamento por senha.
Restaurante	Fluxo organizado, menor sobrecarga no caixa, controle de pedidos pagos e pendentes, cardápio dinâmico e expansão para múltiplos dispositivos.
Equipe técnica	Arquitetura modular, segurança por perfil/dispositivo, pagamento desacoplado e evolução incremental do MVP.

3.3 Objetivos específicos

- Criar cardápio digital interativo.
- Permitir montagem e personalização de pedidos.
- Registrar pedidos no backend.
- Separar pedido de pagamento.
- Confirmar pagamentos antes do envio à cozinha.
- Criar painéis de caixa, cozinha e administração.
- Proteger a API com autenticação, autorização e dispositivos cadastrados.
- Preparar o sistema para integrações reais de pagamento.

4. Escopo do MVP

Incluído no MVP	Deixar para evolução futura
Cadastro de restaurante, login administrativo e gestão básica de usuários.	Pix real, TEF, SmartPOS e integração com maquininha.
Cadastro de categorias, produtos, destaques e disponibilidade.	Nota fiscal, obrigações fiscais e integrações governamentais.
Totem/PWA com cardápio, carrinho, pedido e confirmação.	Relatórios avançados, estoque e promoções complexas.
Pagamento simulado e dinheiro com confirmação no caixa.	mTLS, 2FA e segurança avançada.
Cadastro de dispositivos, token por dispositivo e auditoria básica.	Multiunidade avançada e microserviços.
Painel do caixa e painel da cozinha.	Impressora térmica e comanda automática.

5. Atores, perfis e dispositivos

5.1 Usuários humanos

Perfil	Descrição	Permissões principais
--------	-----------	-----------------------

SUPER_ADMIN	Usuário da empresa desenvolvedora.	Gerenciar restaurantes, dispositivos, cardápios, configurações e auditoria.
ADMIN_RESTAURANTE	Responsável pelo restaurante.	Gerenciar cardápio, preços, categorias, produtos e disponibilidade.
OPERADOR_CAIXA	Funcionário do caixa.	Confirmar pagamento, cancelar pedidos pendentes e consultar pedidos.
OPERADOR_COZINHA	Funcionário da cozinha.	Visualizar pedidos pagos e alterar status de preparo.

5.2 Dispositivos cadastrados

Tipo de dispositivo	Permissões sugeridas	Restrições
TOTEM	Ver cardápio, criar pedido, iniciar pagamento e consultar status do próprio pedido.	Não altera produtos, não confirma dinheiro, não acessa admin.
CAIXA	Ver pendências, confirmar pagamento, cancelar pedido pendente.	Não altera cardápio sem perfil administrativo.
COZINHA	Ver pedidos pagos e alterar status de preparo.	Não acessa pagamentos ou dados administrativos.
ADMINISTRACAO	Gerenciar produtos, categorias, imagens, preços, usuários e dispositivos.	Exige perfil administrativo.

6. Diagrama de casos de uso

O diagrama de casos de uso representa as interações centrais entre atores e funcionalidades. Ele orienta a definição de telas, endpoints e permissões de acesso.

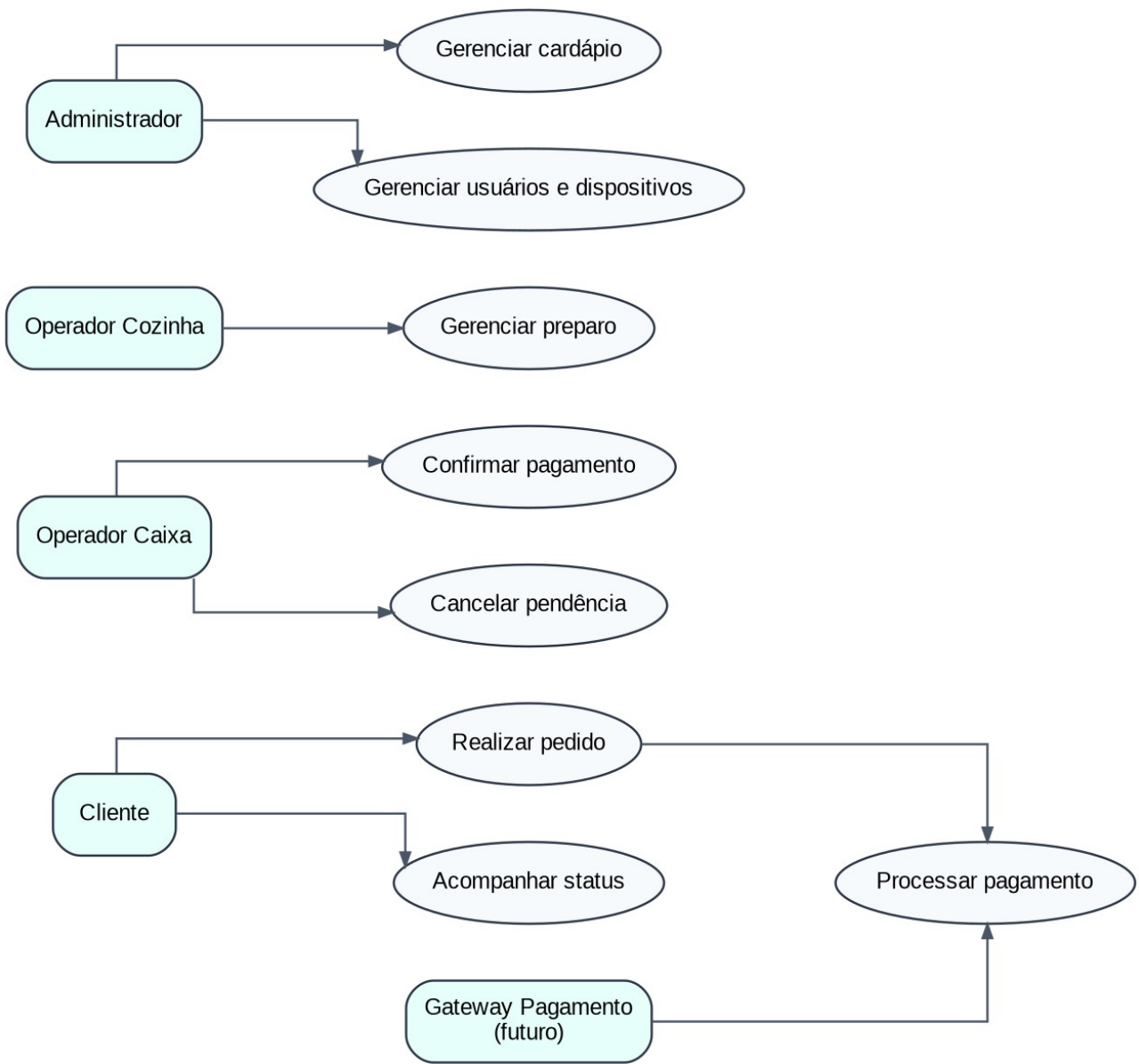


Figura 2 - Diagrama de casos de uso do sistema.

6.1 Casos de uso principais

Módulo	Casos de uso
Totem	Iniciar atendimento, visualizar cardápio, selecionar produto, personalizar pedido, gerenciar carrinho, escolher pagamento, confirmar pedido e acompanhar status.
Caixa	Visualizar pedidos pendentes, confirmar dinheiro, cancelar pendência, reimprimir comprovante e consultar histórico.
Cozinha	Visualizar fila, consultar itens e complementos, iniciar preparo, marcar pedido como pronto e finalizar retirada.
Admin	Gerenciar categorias, produtos, dispositivos, usuários, permissões e auditoria.

7. Arquitetura geral e camadas

A arquitetura recomendada é modular, com separação clara entre apresentação, API, regras de negócio, persistência, segurança e integrações. Para o MVP, a comunicação pode ser via REST API. Futuramente, WebSocket pode ser adotado para atualização em tempo real.

Diagrama de Arquitetura em Camadas

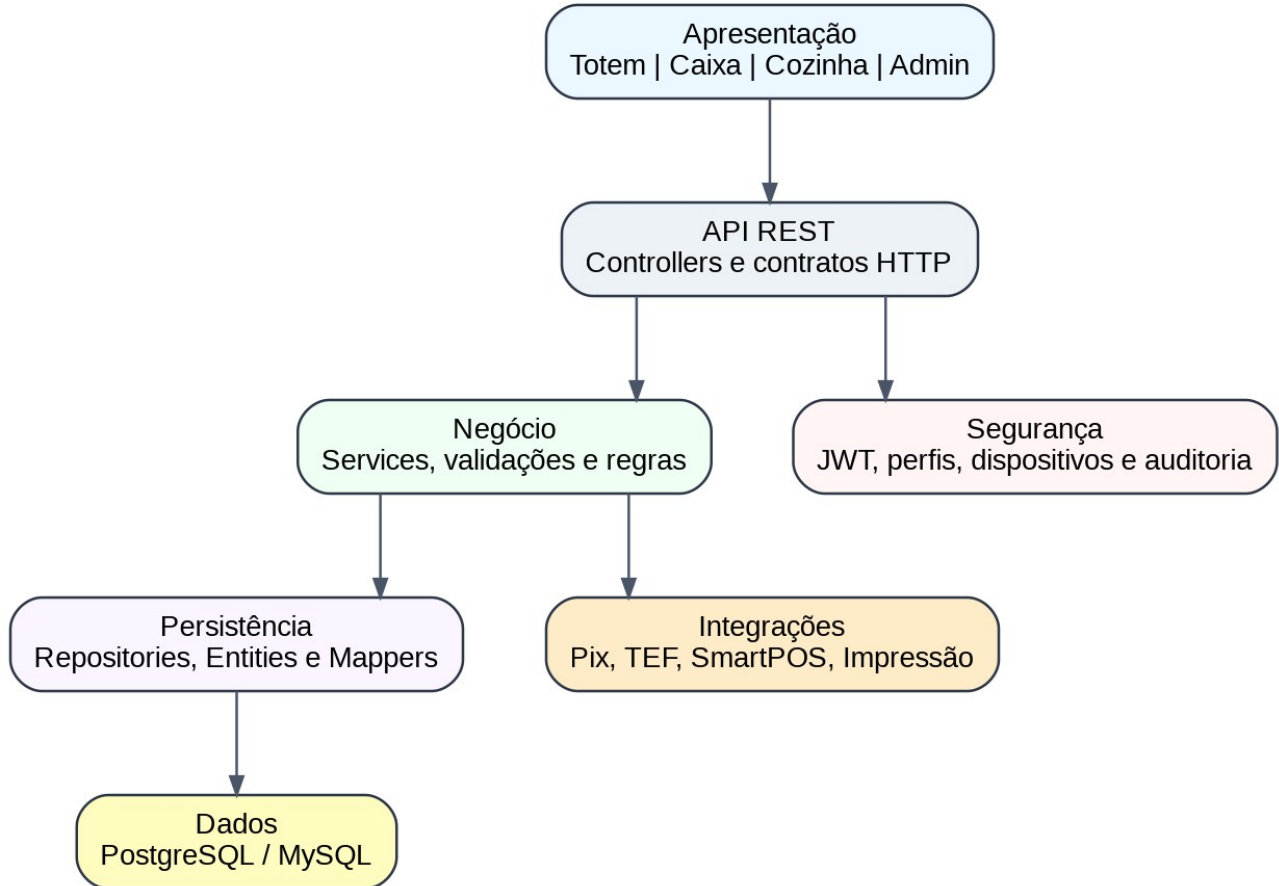


Figura 3 - Arquitetura em camadas recomendada.

7.1 Stack técnica recomendada

Camada	Tecnologias
Backend	Java 21, Spring Boot, Spring Web, Spring Data JPA, Bean Validation, Spring Security.
Banco de dados	PostgreSQL recomendado; MySQL como alternativa.
Segurança	JWT, BCrypt, refresh token revogável, perfis, permissões e dispositivos cadastrados.
Documentação	Swagger/OpenAPI.
Frontend	Angular ou React com TypeScript.
Interface do totem	Web/PWA em tela cheia, com evolução para Android WebView ou Kiosk Mode.
Integrações futuras	Pix, TEF, SmartPOS, gateway de pagamento, impressão e WebSocket.

8. Segurança e controle de acesso

A API não deve ficar aberta para qualquer máquina da rede. Toda requisição relevante deve identificar quem está acessando, se é usuário ou dispositivo, qual restaurante está vinculado, qual perfil possui e se a ação solicitada é permitida.

O controle por IP pode ser usado como camada adicional, mas não deve ser a única proteção. O modelo recomendado combina IP permitido, token válido, dispositivo cadastrado, permissão por função e auditoria.

Fluxo de Validação de Acesso à API

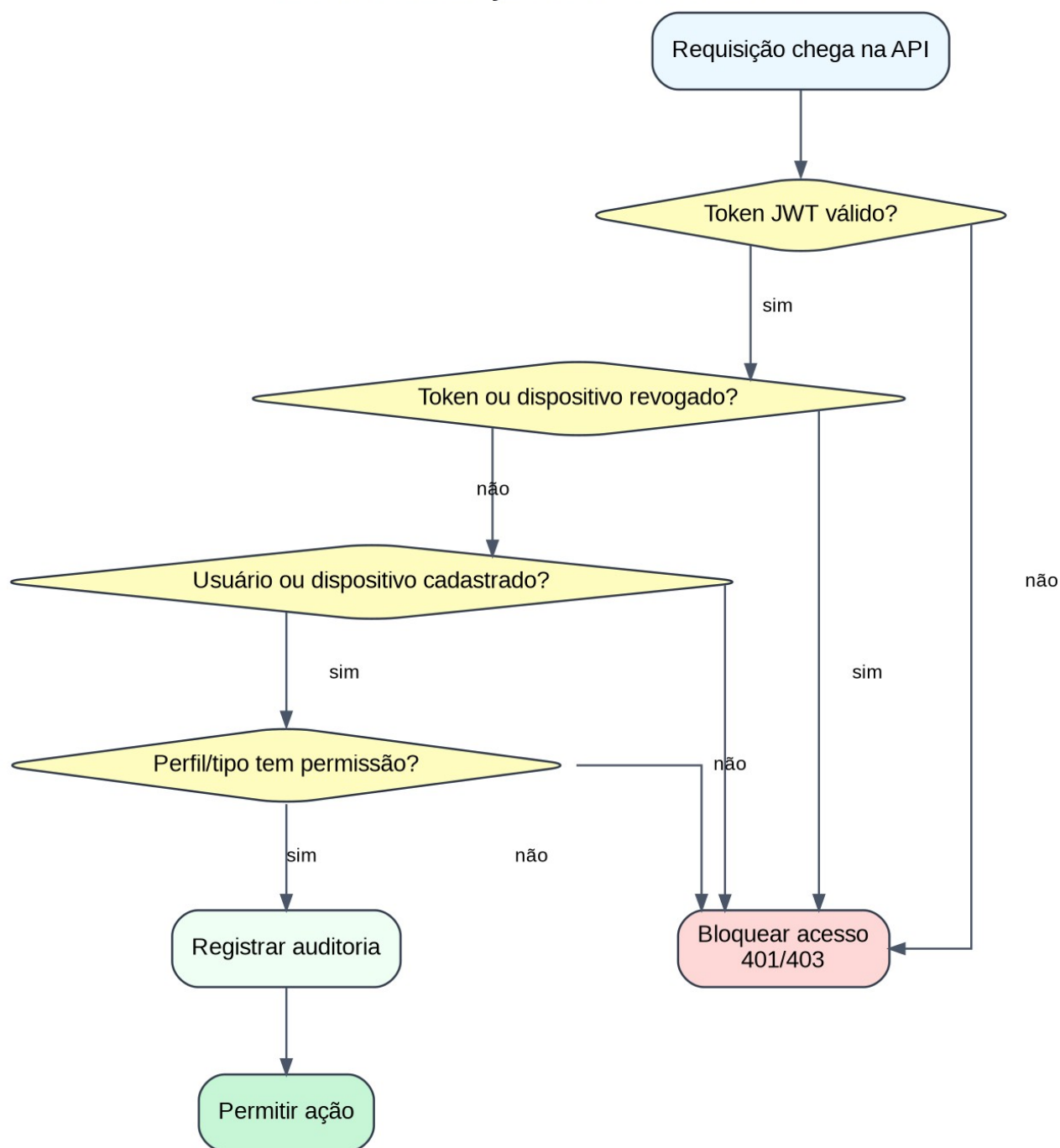


Figura 4 - Fluxo de validação de acesso à API.

8.1 Sessão longa, renovável e revogável

- Evitar sessão literalmente infinita.
- Usar access token de curta duração.
- Usar refresh token de longa duração.
- Permitir renovação automática para totem, caixa e cozinha.
- Permitir revogação pelo administrador em caso de troca, roubo ou comprometimento do equipamento.

9. Ativação de dispositivos

A ativação de dispositivo simplifica a instalação. O administrador cadastra o equipamento, o sistema gera um código de ativação e o responsável informa esse código no tablet ou totem. Após validação, o dispositivo recebe credencial própria e passa a acessar somente os endpoints permitidos para seu tipo.



Figura 5 - Fluxo de ativação de dispositivo.

10. Módulos do sistema

Módulo	Responsabilidades
Totem do Cliente	Boas-vindas, escolha de consumo, cardápio, detalhe de produto, adicionais, carrinho, identificação, pagamento, status e comprovante.
Backend Java	Centraliza regras de negócio, pedidos, produtos, pagamentos, usuários, dispositivos, autenticação, autorização e auditoria.
Painel do Caixa	Visualiza pedidos pendentes, confirma dinheiro, cancela pedido não pago, reimprime comprovante e consulta histórico.
Painel da Cozinha	Visualiza pedidos pagos, itens, adicionais e observações; altera status para em preparo, pronto e retirado.
Painel Administrativo	Gerencia cardápio, produtos, preços, imagens, usuários, dispositivos, permissões e auditoria.
Módulo de Pagamento	Separa o domínio de pagamento do pedido e permite evoluir de FakePaymentProvider para Pix, TEF ou SmartPOS.
Auditoria	Registra ações administrativas, alterações de status, tentativas relevantes e origem da requisição.

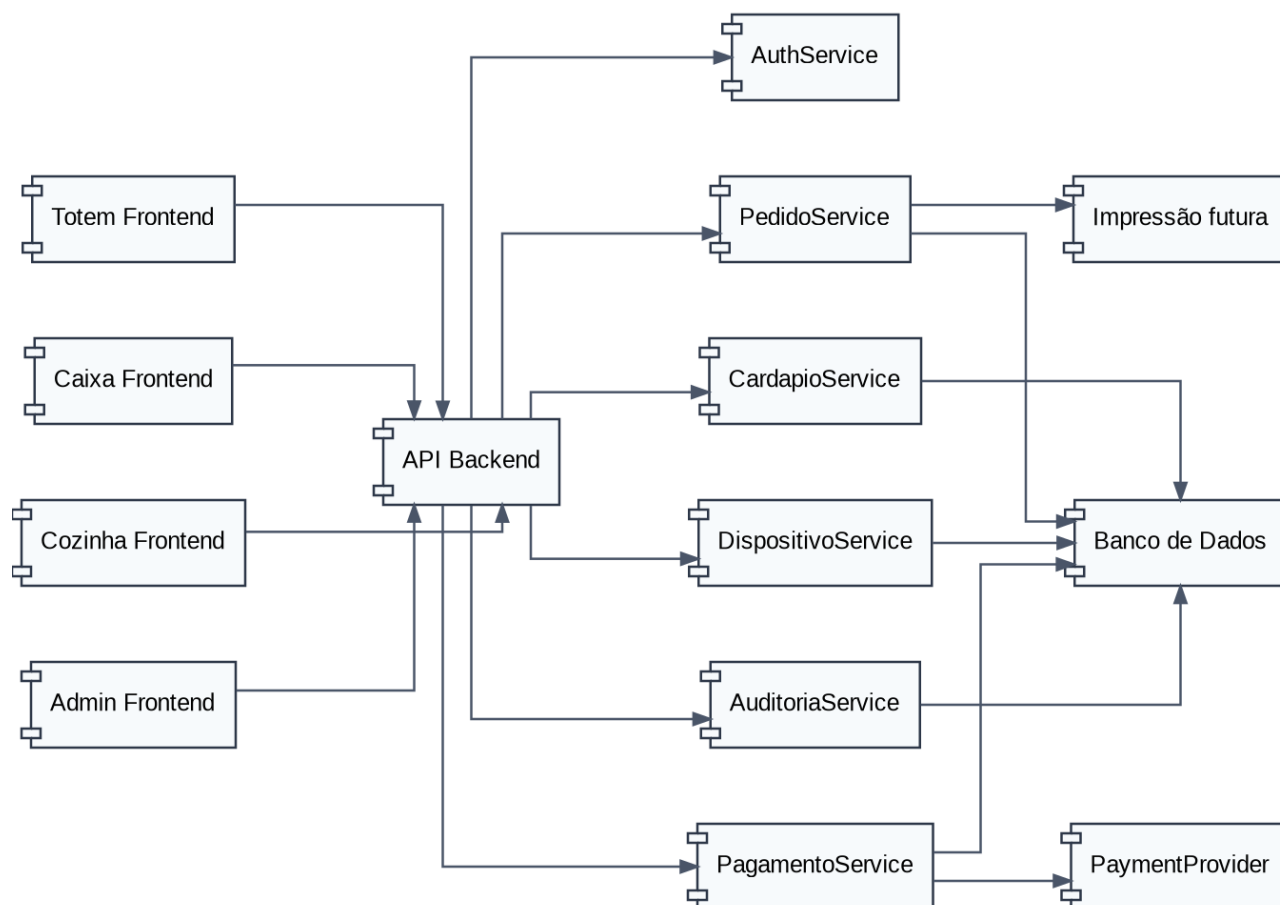


Figura 6 - Diagrama UML de componentes.

11. Modelagem UML e banco de dados

11.1 Diagrama UML de classes

O domínio principal é centrado em Restaurante, Produto, Categoria, Pedido, ItemPedido, Pagamento, Usuário, Dispositivo, RefreshToken, Histórico de Status e Auditoria.

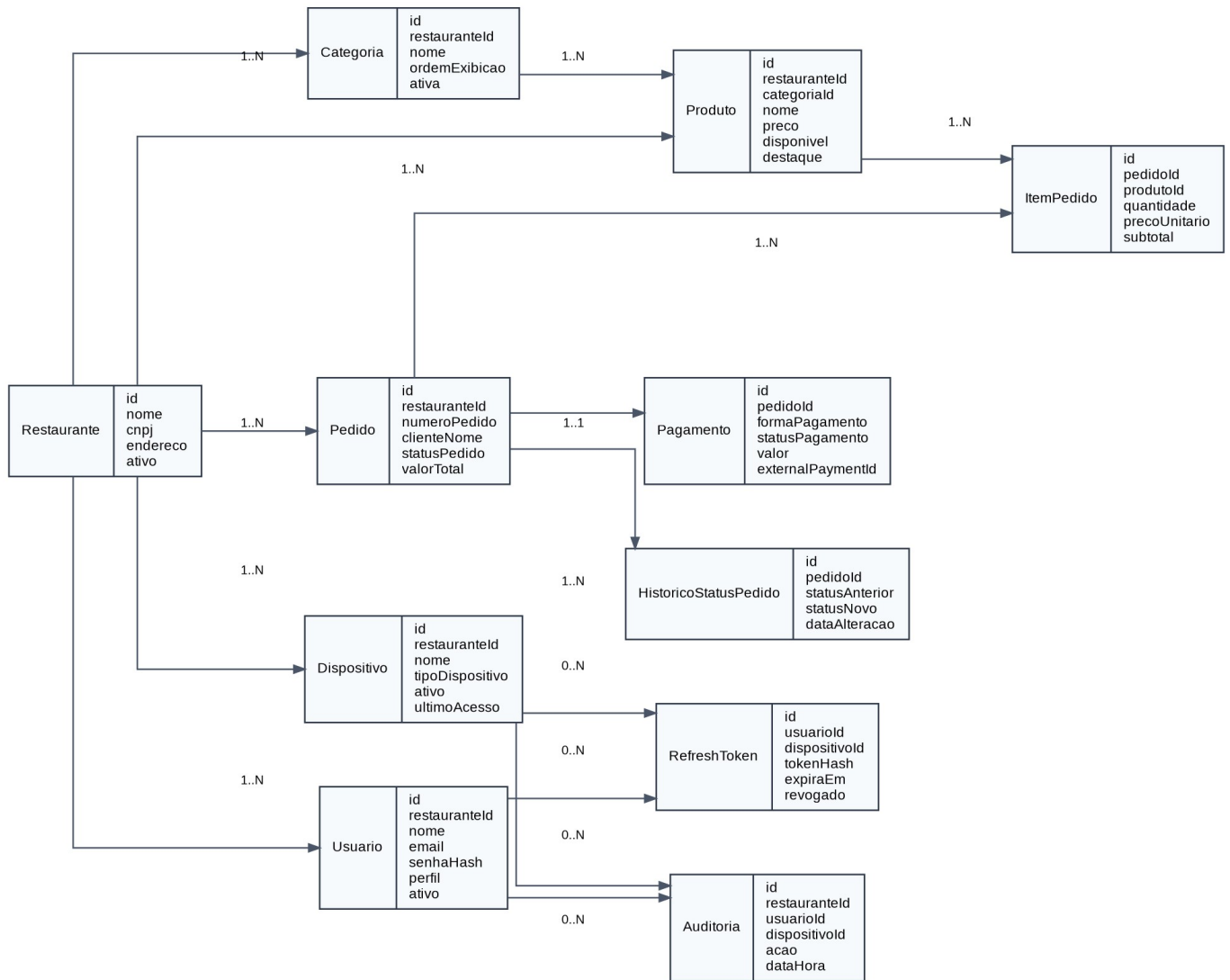


Figura 7 - Diagrama UML de classes do domínio principal.

11.2 Entidades principais e campos

Entidade	Campos principais
Restaurante	id, nome, cnpj, endereco, ativo, criadoEm, atualizadoEm
Produto	id, restauranteId, nome, descricao, preco, imagemUrl, disponivel, destaque, recomendado, ordemExibicao, categoriaId
Categoria	id, restauranteId, nome, descricao, ordemExibicao, ativa
Pedido	id, restauranteId, numeroPedido, clienteNome, tipoConsumo, statusPedido, valorTotal, dispositivoOrigemId
ItemPedido	id, pedidoId, produtoId, nomeProduto, quantidade, precoUnitario, subtotal, observacao
Pagamento	id, pedidoId, formaPagamento, statusPagamento, valor, paymentProvider, externalPaymentId, qrCodePix
Usuario	id, restauranteId, nome, email, senhaHash, perfil, ativo
Dispositivo	id, restauranteId, nome, codigoIdentificacao, tipoDispositivo, ativo, ultimoAcesso
RefreshToken	id, usuarioId, dispositivoId, tokenHash, expiraEm, revogado

HistoricoStatusPedido	id, pedidoId, statusAnterior, statusNovo, dataAlteracao, alteradoPorUsuarioId, alteradoPorDispositivoId
Auditoria	id, restauranteId, usuarioId, dispositivoId, acao, entidadeAfetada, entidadeId, dataHora, ipOrigem, detalhes

11.3 Modelo conceitual de banco de dados

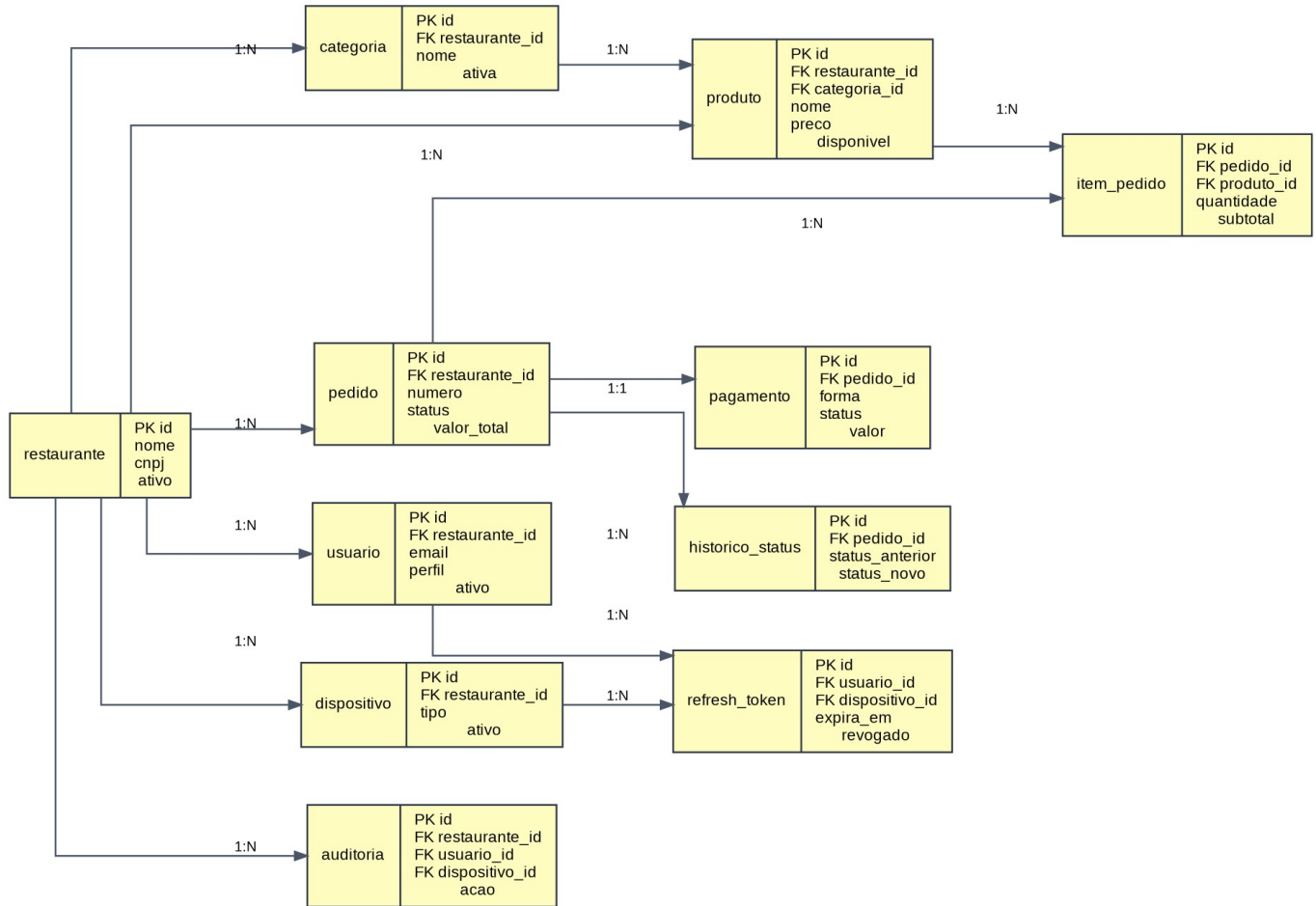


Figura 8 - Modelo entidade-relacionamento conceitual.

12. Estados, enums e regras de negócio

12.1 Diagrama de estados do pedido

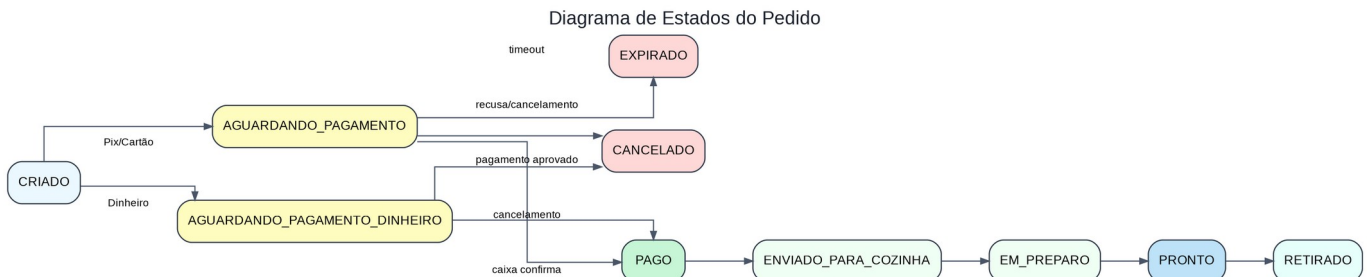


Figura 9 - Diagrama de estados do pedido.

12.2 Enums sugeridos

```
public enum StatusPedido {  
  
    CRIADO,  
  
    AGUARDANDO_PAGAMENTO,  
  
    AGUARDANDO_PAGAMENTO_DINHEIRO,  
  
    PAGO,  
  
    ENVIADO_PARA_COZINHA,  
  
    EM_PREPARO,  
  
    PRONTO,  
  
    RETIRADO,  
  
    CANCELADO,  
  
    EXPIRADO  
}
```

```
public enum StatusPagamento {  
  
    PENDENTE,  
  
    AUTORIZADO,  
  
    RECUSADO,  
  
    CANCELADO,  
  
    ESTORNADO  
}
```

```
public enum FormaPagamento {  
  
    PIX,  
  
    CARTAO_CREDITO,  
  
    CARTAO_DEBITO,  
  
    DINHEIRO  
}
```

```
public enum TipoDispositivo {
```

```
TOTEM,  
  
CAIXA,  
  
COZINHA,  
  
ADMINISTRACAO  
  
}
```

```
public enum PerfilUsuario {  
  
    SUPER_ADMIN,  
  
    ADMIN_RESTAURANTE,  
  
    OPERADOR_CAIXA,  
  
    OPERADOR_COZINHA  
  
}
```

12.3 Regras de negócio críticas

1. Um pedido deve possuir pelo menos um item.
2. Um pedido deve ter valor total maior que zero.
3. Um pedido só pode ir para a cozinha após pagamento confirmado.
4. Pedido em dinheiro deve aguardar confirmação do caixa.
5. Pedido com pagamento recusado não deve ser enviado à cozinha.
6. Pedido não pago dentro de determinado tempo pode expirar.
7. Produto indisponível não deve aparecer para compra no totem.
8. Alterações no pedido devem ser bloqueadas após pagamento confirmado.
9. Todo status importante deve ser registrado em histórico.
10. Totem só pode criar pedidos e consultar informações permitidas.
11. Caixa só pode confirmar ou cancelar pedidos dentro de seu escopo.
12. Cozinha só pode visualizar pedidos pagos e alterar status de preparo.
13. Administração pode alterar cardápio sem alterar código.
14. Toda alteração administrativa relevante deve gerar auditoria.
15. Dispositivo revogado não pode continuar acessando a API.
16. Segurança não deve depender apenas do IP da máquina.

13. Pagamento e estratégia de integração

Pedido e pagamento devem ser tratados separadamente. A regra central é que nenhum pedido deve ser enviado para a cozinha antes da confirmação do pagamento.

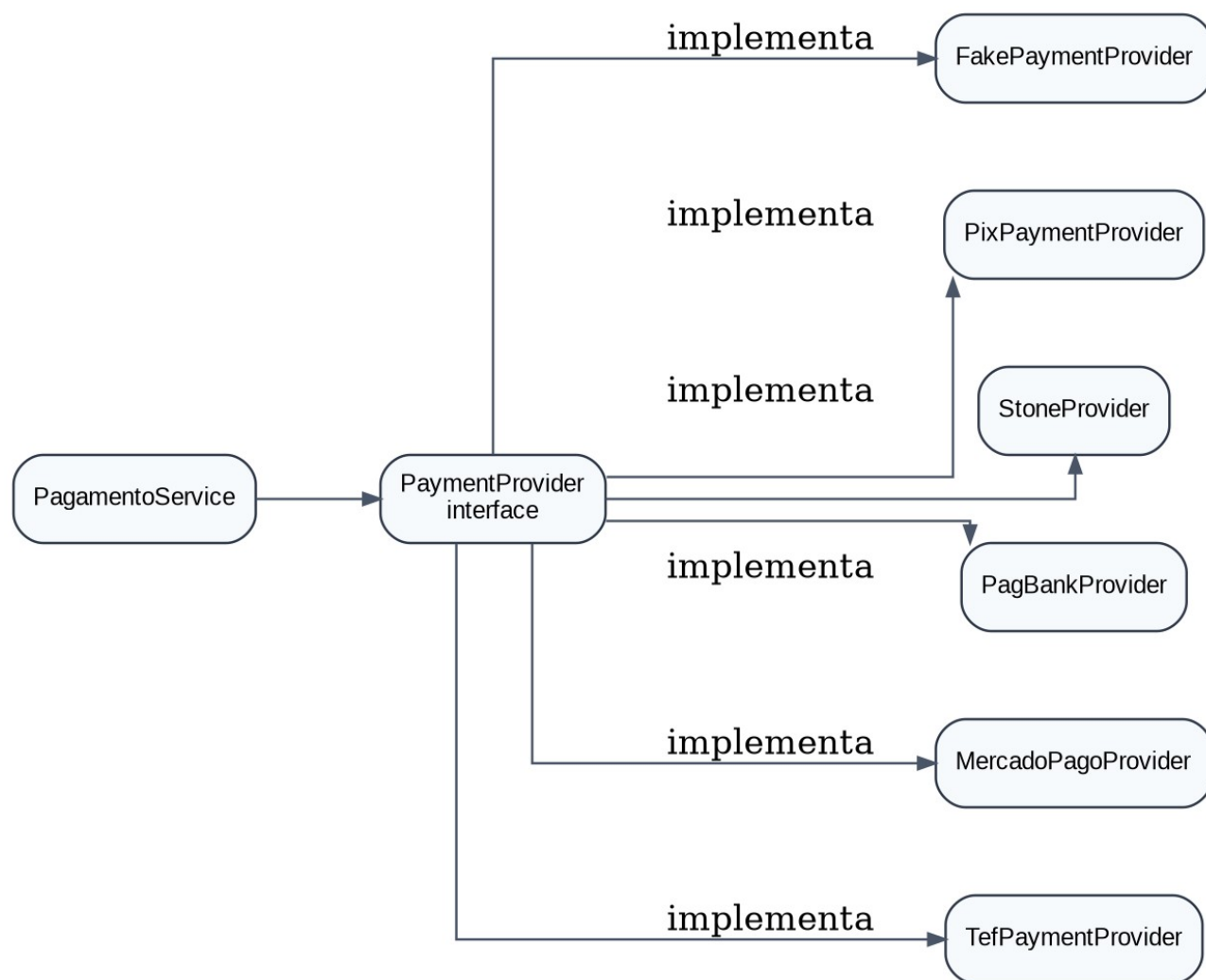


Figura 10 - Estratégia de pagamento desacoplada com PaymentProvider.

13.1 Interface conceitual

```

public interface PaymentProvider {

    PaymentResponse iniciarPagamento(PaymentRequest request);

    PaymentStatus consultarStatus(String externalPaymentId);

    void cancelarPagamento(String externalPaymentId);

}

```

13.2 Modelos de pagamento

Modelo	Descrição	Uso recomendado
FakePaymentProvider	Simula aprovação, recusa e pendência.	MVP e testes de fluxo.
Pix	Gera QR Code, recebe webhook e confirma pagamento.	Primeira integração real recomendada.
Cartão simulado	Sistema aprova/recusa manualmente para testes.	MVP.
Maquininha externa	Funcionário cobra fora do sistema e	Intermediário.

	confirma no caixa.	
SmartPOS/SDK	App chama SDK da adquirente no Android.	Evolução com hardware.
TEF/Pin Pad	Backend/PDV conversa com módulo TEF.	Operação mais profissional.

14. Fluxogramas e diagramas de sequência

14.1 Fluxo de pedido com Pix

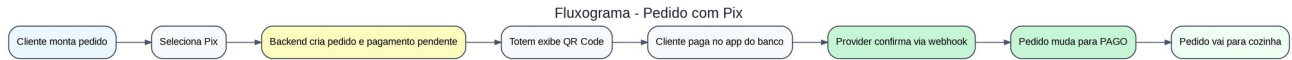


Figura 11 - Fluxograma de pedido com Pix.

14.2 Fluxo de pedido com dinheiro

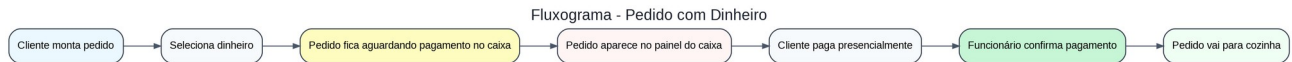


Figura 12 - Fluxograma de pedido com dinheiro.

14.3 Fluxo da cozinha

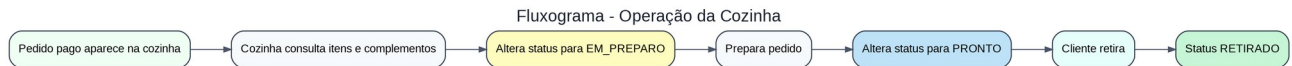


Figura 13 - Fluxograma operacional da cozinha.

14.4 Alteração de cardápio

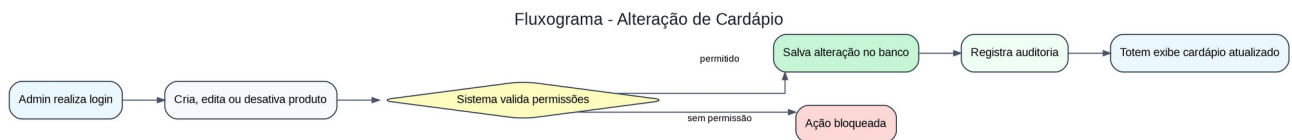


Figura 14 - Fluxograma de alteração de cardápio com auditoria.

14.5 Diagramas de sequência

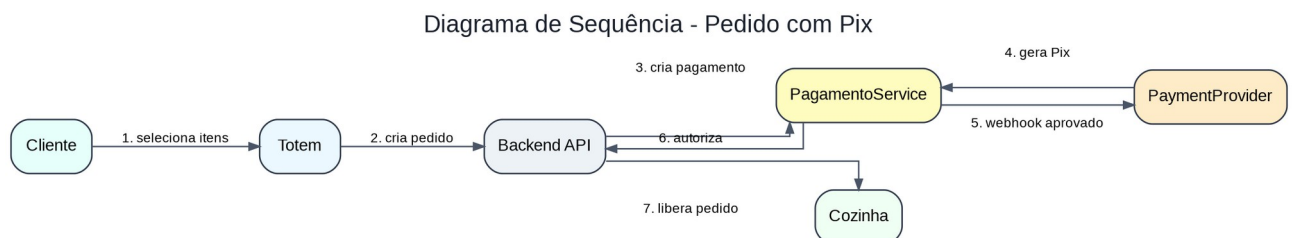


Figura 15 - Diagrama de sequência simplificado: pedido com Pix.

Diagrama de Sequência - Pedido com Dinheiro

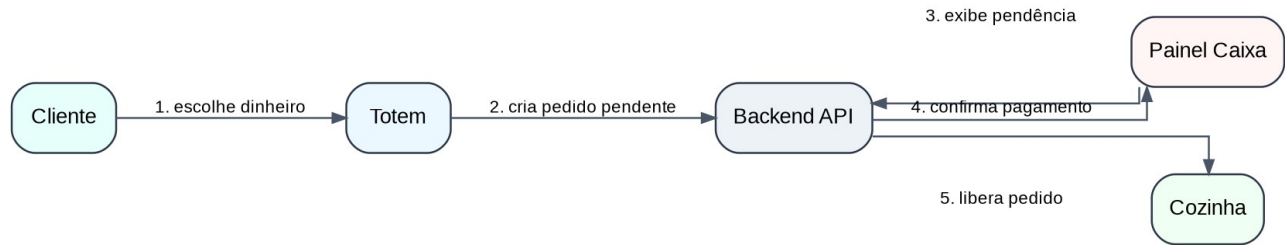


Figura 16 - Diagrama de sequência simplificado: pedido com dinheiro.

Diagrama de Sequência - Ativação de Dispositivo

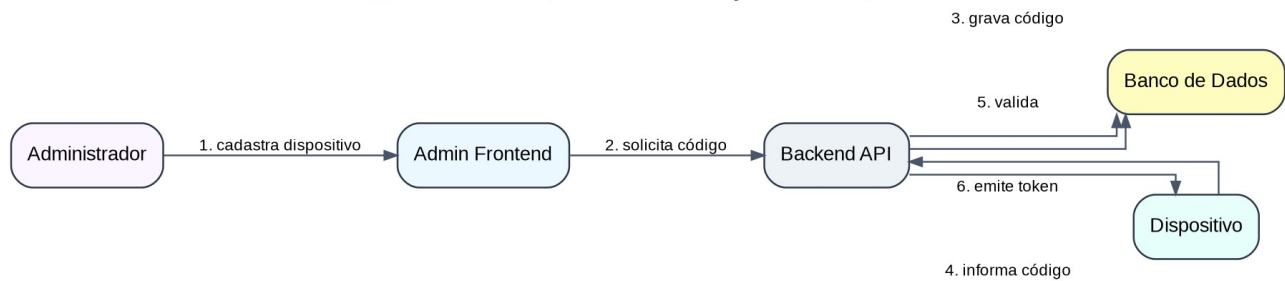


Figura 17 - Diagrama de sequência simplificado: ativação de dispositivo.

15. Organização recomendada do backend

A organização em pacotes deve favorecer separação de responsabilidades, manutenção e evolução. A estrutura abaixo é adequada para uma arquitetura em camadas no Spring Boot.

```
src/main/java/com/projeto/totem
```

```

├── controller
|   ├── admin
|   ├── auth
|   ├── caixa
|   ├── cozinha
|   └── totem
├── service
├── repository
├── entity
├── dto
├── enums
└── exception
    
```

- └─ config
- └─ security
- └─ integration
- | └─ payment
- └─ mapper

Camada	Responsabilidade
controller	Receber requisições HTTP e acionar services.
service	Aplicar regras de negócio, validações e orquestrações.
repository	Acessar o banco de dados via Spring Data JPA.
entity	Representar tabelas e relacionamentos.
dto	Transportar dados entre API e cliente sem expor entidades.
enums	Padronizar status, formas de pagamento, tipos e perfis.
exception	Tratamento global e padronizado de erros.
config	CORS, Swagger, beans e configurações gerais.
security	Autenticação, autorização, JWT, filtros e permissões.
integration	Integrações externas, como pagamentos e impressão.
mapper	Conversão entre Entity e DTO.

16. Endpoints REST iniciais

16.1 Autenticação e segurança

Método	Rota	Objetivo
POST	/api/auth/login	Autenticar usuário humano.
POST	/api/auth/refresh	Renovar sessão com refresh token.
POST	/api/auth/logout	Encerrar sessão e revogar token.
POST	/api/auth/dispositivos/ativar	Ativar dispositivo por código.

16.2 Totem

Método	Rota	Objetivo	Permissão
GET	/api/totem/cardapio	Listar cardápio disponível.	DEVICE_TOTEM
POST	/api/totem/pedidos	Criar pedido.	DEVICE_TOTEM
GET	/api/totem/pedidos/{id}	Consultar pedido.	DEVICE_TOTEM
POST	/api/totem/pedidos/{id}/pagamento	Iniciar pagamento.	DEVICE_TOTEM

16.3 Caixa, cozinha e administração

Módulo	Método	Rota	Objetivo
Caixa	GET	/api/caixa/pedidos/pendentes	Listar pendências de pagamento.
Caixa	POST	/api/caixa/pedidos/{id}/confirmar-pagamento	Confirmar dinheiro ou pagamento manual.
Caixa	POST	/api/caixa/pedidos/{id}/cancelar	Cancelar pedido pendente.
Cozinha	GET	/api/cozinha/pedidos	Listar pedidos pagos.

Cozinha	PATCH	/api/cozinha/pedidos/{id}/status	Atualizar status de preparo.
Admin	POST	/api/admin/categorias	Criar categoria.
Admin	GET	/api/admin/categorias	Listar categorias.
Admin	PUT	/api/admin/categorias/{id}	Atualizar categoria.
Admin	DELETE	/api/admin/categorias/{id}	Remover/inativar categoria.
Admin	POST	/api/admin/produtos	Criar produto.
Admin	GET	/api/admin/produtos	Listar produtos.
Admin	PUT	/api/admin/produtos/{id}	Atualizar produto.
Admin	PATCH	/api/admin/produtos/{id}/disponibilidade	Alterar disponibilidade.
Admin	PATCH	/api/admin/produtos/{id}/destaque	Alterar destaque.
Admin	POST	/api/admin/dispositivos	Cadastrar dispositivo.
Admin	GET	/api/admin/dispositivos	Listar dispositivos.
Admin	PATCH	/api/admin/dispositivos/{id}/revogar	Revogar dispositivo.
Admin	POST	/api/admin/usuarios	Cadastrar usuário.
Webhook	POST	/api/webhooks/pix	Receber confirmação Pix futura.
Webhook	POST	/api/webhooks/pagamentos	Receber confirmações de provedores futuros.

17. Contratos de dados

17.1 Criar pedido

POST /api/totem/pedidos

```
{
  "tipoConsumo": "LOCAL",
  "clienteNome": "Fabio",
  "itens": [
    {
      "produtoId": 10,
      "quantidade": 1,
      "complementos": [3, 7],
      "observacao": "Sem cebola"
    }
  ]
}
```

17.2 Resposta de pedido criado

HTTP 201 Created

```
{  
  
  "pedidoId": 1024,  
  
  "numeroPedido": "A1024",  
  
  "statusPedido": "CRIADO",  
  
  "valorTotal": 39.90,  
  
  "itens": [  
  
    {  
  
      "produtoId": 10,  
  
      "nomeProduto": "Combo X-Burger",  
  
      "quantidade": 1,  
  
      "precoUnitario": 39.90,  
  
      "subtotal": 39.90  
    }  
  ]  
}
```

17.3 Iniciar pagamento

POST /api/totem/pedidos/1024/pagamento

```
{  
  
  "formaPagamento": "PIX"  
}
```

HTTP 200 OK

```
{  
  
  "pagamentoId": 9001,  
  
  "statusPagamento": "PENDENTE",  
  
  "qrCodePix": "000201...",  
  
  "expiraEm": "2026-05-05T15:30:00"  
}
```

17.4 Confirmar dinheiro

POST /api/caixa/pedidos/1024/confirmar-pagamento

```
{  
  
  "formaPagamento": "DINHEIRO",  
  
  "valorRecebido": 50.00,  
  
  "observacao": "Pagamento recebido no caixa"  
}
```

17.5 Atualizar status na cozinha

PATCH /api/cozinha/pedidos/1024/status

```
{  
  
  "statusPedido": "EM_PREPARO"  
}
```

17.6 Erro padrão

HTTP 409 Conflict

```
{  
  
  "codigo": "PEDIDO_NAO_PAGO",  
  
  "mensagem": "Pedido não pode ser enviado para a cozinha antes da confirmação do pagamento.",  
  
  "campo": "statusPedido",  
  
  "timestamp": "2026-05-05T15:10:22",  
  
  "path": "/api/cozinha/pedidos/1024/status"  
}
```

18. Boas práticas, testes e qualidade

18.1 Boas práticas de desenvolvimento

- Usar DTOs para entrada e saída.
- Validar requests com Bean Validation.
- Não confiar em preço enviado pelo frontend; recalcular tudo no backend.
- Usar tratamento global de exceções.
- Registrar logs estruturados.
- Registrar auditoria em ações sensíveis.
- Versionar banco com Flyway.
- Manter Swagger/OpenAPI atualizado.

- Separar integração de pagamento por interface.
- Evitar acoplamento entre pedido e provedor de pagamento.
- Aplicar controle de acesso por perfil e tipo de dispositivo.

18.2 Testes recomendados

Tipo de teste	Exemplos
Unitários	Services de pedido, pagamento, status, permissões e cálculo de total.
Integração	Controllers, repositories e fluxo com banco de teste.
Segurança	Acesso negado para dispositivo revogado, perfil sem permissão e token expirado.
Fluxo completo	Pedido com Pix, cartão simulado, dinheiro, cozinha e retirada.
Regressão	Produto indisponível, pedido expirado, pagamento recusado e alteração após pagamento.

19. Comprovante, comanda, nota fiscal e modo totem

19.1 Comprovante no MVP

Na primeira versão, recomenda-se emitir apenas comprovante interno de pedido, comanda ou senha de retirada. Evitar chamar de nota fiscal no MVP, pois nota fiscal real envolve obrigações fiscais, integrações fiscais e autorização governamental.

PEDIDO #1024

Cliente: Fabio

Itens:

1x Combo X-Burger

1x Batata Média

1x Refrigerante

Total: R\$ 39,90

Pagamento: Pix

Status: Pago

Retirada no balcão

19.2 Controle do tablet e modo totem

Opção	Descrição	Recomendação
Web/PWA	Aplicação web em tela cheia.	Melhor para MVP.
Android WebView	App Android abrindo a aplicação web.	Boa evolução futura.
Android nativo	Aplicativo em Java/Kotlin.	Útil para SDK de maquininha.
Kiosk Mode	Tablet travado em um único app.	Ideal para operação real.

20. Roadmap, riscos e decisões pendentes

20.1 Roadmap de desenvolvimento



Figura 18 - Roadmap visual de desenvolvimento do MVP e evoluções.

20.2 Riscos e mitigações

Risco	Impacto	Mitigação
Integração com maquininha ser complexa	Pode atrasar o projeto.	Começar com pagamento simulado.
Pix exigir provedor externo	Depende de API/gateway.	Preparar PaymentProvider.
Internet cair	Afeta pagamentos e sincronização.	Avaliar operação local futura.
Pedido ir para cozinha sem pagamento	Prejuízo operacional.	Bloquear envio até pagamento confirmado.
Tablet permitir sair do sistema	Risco de uso indevido.	PWA fullscreen ou Kiosk Mode.
Nota fiscal real ser complexa	Escopo fiscal elevado.	Começar com comprovante interno.
API ficar aberta na rede	Acesso indevido.	Autenticação, autorização e token por dispositivo.
Sessão infinita comprometida	Acesso indevido prolongado.	Refresh token revogável.
Funcionário alterar cardápio sem permissão	Erro operacional.	Perfis e permissões administrativas.
Produto alterado sem rastreio	Falta de controle.	Auditoria de alterações.

20.3 Perguntas em aberto

- O sistema será local, em nuvem ou ambos?
- O primeiro frontend será Angular ou React?
- O Pix real entra em qual fase?
- O cartão será simulado ou confirmado manualmente?
- Haverá cadastro de funcionários desde o MVP?
- Produtos terão adicionais obrigatórios e opcionais?
- Haverá controle de estoque?
- Haverá impressão de comanda?
- O sistema será multi-restaurantes desde o início?
- Qual será a validade do token do dispositivo?
- O IP será usado como camada adicional de segurança?

21. Conclusão técnica

O projeto é viável e possui boa aplicação prática. A recomendação é iniciar com um MVP funcional, com backend Java/Spring Boot, segurança básica, login administrativo, controle de dispositivos, pagamento simulado, painel do caixa, painel da cozinha e interface PWA para o totem.

Os três pilares técnicos do sistema são pedido, pagamento e segurança. A separação entre esses domínios reduz risco, facilita testes e permite evoluir para integrações reais sem reescrever o núcleo do sistema.

- Começar com segurança inicial e cardápio administrativo.
- Criar cadastro e ativação de dispositivos.
- Implementar pedido e pagamento simulado.
- Liberar fluxo de caixa e cozinha.
- Criar interface do totem.
- Evoluir para Pix real, maquininha, comanda e modo kiosk.